ADITYA UNIVERSITY

School of Computing

Department of Information Technology

OBJECT ORIENTED ANALYSIS AND DESIGN USING UML

A Cornerstone Project Report on

"SMART UNIVERSITY ERP"

Submitted by

SAI HARINI SINGINEEDI(24B11IT097)

NALLI PRANEESHA (24B11IT072)

PEDAPUDI SNEHALATHA (24B11IT083)

MURALASETTI ANANTHA LAKSHMI (25B21IT007)

Under the guidance of

Mrs. J. Divya Lalitha Sri

Department of Information Technology

ADITYA UNIVERSITY, Surampalem

Academic Year 2026-2027

ADITYA UNIVERSITY

School of Computing

Department of Information Technology

OBJECT ORIENTED ANALYSIS AND DESIGN USING UML

CERTIFICATE

This is to certify that the Cornerstone project work entitled “SMART UNIVERSITY ERP” is submitted by

SAI HARINI SINGINEEDI(24B11IT097)

NALLI PRANEESHA(24B11IT072)

PEDAPUDI SNEHALATHA(24B11IT083)

MURALASETTI ANANTHA LAKSHMI(25B21IT007)

in partial fulfilment of the requirements for the award of the B.Tech. degree in Information Technology during the academic year 2026-2027.

Project Guide	**Head of the Department**
Mrs. J. Divya Lalitha Sri	Dr. M. Raja Babu
Department of IT	Department of IT

DECLARATION

We hereby declare that the project entitled “SMART UNIVERSITY ERP” is our original work and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfilment of the requirements for the award of the B.Tech. degree in Information Technology. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We also confirm that this work has been carried out with academic honesty and complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem

Date:

By

SAI HARINI SINGINEEDI(24B11IT097)

NALLI PRANEESHA(24B11IT072)

PEDAPUDI SNEHALATHA(24B11IT083)

MURALASETTI ANANTHA LAKSHMI(25B21IT007)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, Mrs. J. Divya Lalitha Sri, Assistant Professor, who has guided us throughout the project. Their valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We are grateful to Dr. M. Raja Babu, Head of the Department, for inspiring us all the way and for arranging all the facilities and resources needed for our project. We wish to thank Dr. M. V. Rajesh, Associate Dean (School of Computing), Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), and Dr. G. Suresh, Registrar, for their encouragement and support during the course of our project.

We would like to extend our sincere thanks to Dr. M. B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, and the Management, Aditya University, for their unconditional support in providing us with the best infrastructural facilities and state-of-the-art laboratories during our project.

Not to forget, the non-teaching staff and our friends, who have directly or indirectly supported us in completing this project on time.

ABSTRACT

The Smart University ERP System is a software application designed to automate core university and academic operations. Students can register, log in, view courses, access timetables, check attendance, view examination schedules and results, pay fees, and track academic progress. Faculty members manage student attendance, assignments, marks, and course materials, while administrators manage student records, faculty details, departments, courses, fees, examinations, and reports.

This project applies Object-Oriented Analysis and Design (OOAD) principles and standard UML models to represent the actors, system structure, interactions, workflows, and academic lifecycle. The model covers use case, class, sequence, communication, activity, state machine, package, component, and deployment diagrams, and applies the Boundary–Control–Entity (BCE) layering to the interaction diagrams. The resulting design provides a structured foundation for implementation and for mapping the class model to a relational database.

The report presents the requirement analysis, the use case and domain models, the interaction models, the design class model, the behavioural and architectural models, and the database design. It also records the tools used and the limitations of the current model, and suggests future enhancements such as implementing the student management and attendance modules first, integrating online fee payments, and extending the system to support library management, transport, hostel facilities, and parent access.

Keywords: Smart University ERP, OOAD, UML, Use Case Diagram, Class Diagram, Sequence Diagram, Activity Diagram, State Machine Diagram, Boundary–Control–Entity.

TABLE OF CONTENTS

S.No.	Contents	Pages
1	Title Page	i
2	Certificate	ii
3	Declaration	iii
4	Acknowledgement	iv
5	Abstract	v
6	Table of Contents	vi
7	List of Figures	viii
8	List of Tables	ix
Chapter 1	**INTRODUCTION**	**1**
1.1	Introduction to the Project	1
1.2	Project Modules	2
1.3	Problem Statement	2
1.4	Objectives of the Project	3
1.5	Scope of the Project	4
Chapter 2	**REQUIREMENT ANALYSIS**	**5**
2.1	Functional Requirements	5
2.2	Non-Functional Requirements	6
2.3	Actors and Their Roles	7
2.4	Use Case Identification	8
2.5	Use Case Descriptions	9
2.6	Event List	13
2.7	Event Table	14
Chapter 3	**USE CASE AND DOMAIN MODELLING**	**16**
3.1	Use Case Modelling	16
3.1.1	Use Case Diagram	16
3.1.2	Description and Terminology	16
3.1.3	Project-Specific Explanation	17
3.2	Domain Modelling	18
3.2.1	Domain Class Diagram	18
3.2.2	Description and Terminology	18
3.2.3	Project-Specific Explanation	19
Chapter 4	**INTERACTION MODELLING**	**21**
4.1	System Sequence Diagram	21
Chapter 5	**DESIGN CLASS MODELLING**	**22**
5.1	Design Class Diagram	22
5.2	Classes, Attributes and Operations	22
5.3	Relationships and Multiplicity	24
5.4	Interfaces, Dependencies and Generalization	24
Chapter 6	**BEHAVIOURAL MODELLING**	**26**

S.No.	Contents	Pages
6.1	Activity Diagram	26
6.2	State Machine Diagram	27
Chapter 7	**ARCHITECTURAL MODELLING**	**29**
7.1	Package Diagram	29
7.2	Component Diagram	30
7.3	Deployment Diagram	32
Chapter 8	**DATABASE DESIGN**	**34**
8.1	Mapping Class Diagram to Relational Database	34
8.2	Database Tables	34
8.3	Primary Keys and Foreign Keys	37
8.4	CRUD Matrix	39
Chapter 9	**TOOLS AND TECHNOLOGIES USED**	**41**
9.1	Overview of Tools and Technologies	41
9.2	UML Modelling Tool	41
9.3	Introduction to the Tool	42
9.4	Features and Capabilities	42
Chapter 10	**CONCLUSION AND FUTURE SCOPE**	**43**
10.1	Conclusion	43
10.2	Limitations	43
10.3	Future Enhancements	44

LIST OF FIGURES

Figure No.	Title	Page
Figure 3.1	Use Case Diagram	16
Figure 3.2	Domain Class Diagram	18
Figure 4.1	System Sequence Diagram	21
Figure 5.1	Design Class Diagram	22
Figure 6.1	Activity Diagram	26
Figure 6.2	State Machine Diagram – Student Lifecycle	28
Figure 7.1	Package Diagram	29
Figure 7.2	Component Diagram	31
Figure 7.3	Deployment Diagram	32

LIST OF TABLES

Table No.	Title	Page
Table 2.1	Functional Requirements	5
Table 2.2	Non-Functional Requirements	6
Table 2.3	Actors and Their Roles	7
Table 2.4	Use Case Identification	8
Table 2.5	Use Case Description – Student Registration	9
Table 2.6	Use Case Description – Make Fee Payment	9
Table 2.7	Use Case Description – Cancel Course Registration	10
Table 2.8	Use Case Description – View Attendance	10
Table 2.9	Use Case Description – Update Examination Results	11
Table 2.10	Use Case Description – Assign Faculty to Course	11
Table 2.11	Use Case Description – Check Academic Results	12
Table 2.12	Use Case Description – Manage Student Records	12
Table 2.13	Use Case Description – Schedule Examinations	13
Table 2.14	Event Table	14
Table 5.1	Classes, Attributes and Operations	22
Table 5.2	Relationships and Multiplicity	24
Table 6.1	Actions and Decisions in the Student Registration Activity	27
Table 6.2	Transitions of the Student Lifecycle	28
Table 7.1	Package Dependencies	29
Table 7.2	Components and Their Responsibilities	31
Table 7.3	Nodes and Deployed Artifacts	32
Table 8.1	STUDENT	35
Table 8.2	DEPARTMENT	35
Table 8.3	COURSE	35
Table 8.4	FACULTY	35
Table 8.5	RESULTS	36
Table 8.6	ADMIN	36
Table 8.7	ATTENDANCE	36
Table 8.8	EXAM	37
Table 8.9	FEES	37
Table 8.10	LIBRARY	37
Table 8.11	Primary Keys and Foreign Keys	37
Table 8.12	CRUD Matrix	39
Table 9.1	Tools Used in the Project	41

CHAPTER 1 – INTRODUCTION

1.1 Introduction to the Project

A university is a complex environment in which students, faculty members, administrative staff, and management must cooperate continuously. Student registration, course allocation, attendance tracking, timetable management, examinations, fee payments, and result processing are all interdependent, and a failure to coordinate any one of them can affect other academic activities.

The Smart University ERP System automates these operations across students, faculty members, and university administration. Students can register, access course details, view timetables, check attendance, submit assignments, view examination results, and pay fees. Faculty members manage attendance, assignments, marks, and course materials, while administrators manage student records, faculty details, departments, courses, fees, examinations, and reports.

The project is carried out as part of the course Object Oriented Analysis and Design (OOAD) using UML. UML diagrams provide a structured, object-oriented view of the requirements, the system structure, and the system behaviour, and form the blueprint from which the application and its database can be implemented.

Object-oriented analysis identifies the real-world objects of the university, such as students, faculty members, courses, departments, attendance records, examinations, fees, and results, together with their properties and behaviour. Object-oriented design then refines these objects into classes, interfaces, and components. Using UML for both steps gives the team, the guide, and the reviewers a common notation, so that every design decision can be traced from a requirement to a diagram.

The use of UML models in this project ensures the following:

- Requirements of the Smart University ERP System are understood in the same way by all members of the team.
- Errors and missing requirements related to student registration, attendance, course management, fee payments, and examination results are identified at the design stage, where they are easier and less expensive to correct.
- Models provide documentation for future maintenance and system enhancements.

1.2 Project Modules

The system is organised into the following modules, which correspond to the packages shown in the package diagram in Chapter 7.

- Student Management: student registration, login, profile management, and academic record access.
- Faculty Management: faculty registration, profile management, course allocation, and student performance tracking.
- Course Management: course details, subject allocation, syllabus management, and department-wise course information.
- Attendance Management: student attendance recording, attendance percentage calculation, and attendance report generation.
- Examination Management: exam scheduling, hall ticket details, marks entry, and result publication.
- Fee Management: fee structure management, online fee payments, payment verification, and fee receipt generation.
- Timetable Management: class schedules, faculty timetables, classroom allocation, and timetable updates.
- Notification Management: notifications for attendance, examinations, assignments, fee payments, results, and important university announcements.

Each module of the Smart University ERP System has a clear responsibility and communicates with other modules through well-defined operations. This keeps the modules loosely coupled, so that a change in one module, for example adding a new online fee payment method, has minimal impact on the other modules.

1.3 Problem Statement

A university needs coordinated management of student registration, faculty details, course allocation, attendance, timetables, examinations, fee payments, assignments, results, and academic reports. When these activities are handled by separate or manual processes, data become inconsistent, students face delays in accessing information, errors occur in attendance and result records, and administrators cannot easily track the current status of academic activities.

An integrated system is therefore required to make these operations consistent and manageable. The problem addressed by this project is to analyse and design a Smart University ERP System using object-oriented principles and UML, so that it can be implemented with a clear structure and a reliable database design.

The main difficulties in the present situation can be summarised as follows:

- Student attendance and academic records are not available in one integrated system.
- Course registration and timetable allocation are not properly coordinated.
- Examination marks, results and fee payment records are maintained separately.
- Communication between students, faculty and administration is delayed.
- Management reports have to be prepared manually from separate records.

1.4 Objectives of the Project

- To identify the actors and major Smart University ERP use cases.
- To design the classes for Student, Faculty, Course, Attendance, Examination, Fee Payment and Administrator.
- To model the dynamic behaviour of the system using sequence, activity, communication and state diagrams.
- To demonstrate the OOAD concepts of inheritance, encapsulation and association.
- To provide a design that can be mapped to a database schema and application code using UML.

On completion of the project the following outcomes are expected:

- A complete set of UML models for the Smart University ERP System
- A class model whose classes, attributes and relationships can be converted into database tables.
- Clear descriptions of student registration, attendance, examination, fee payment and result management flows that a developer can follow.
- A modular architecture in which new courses, departments, faculty members and university services can be added.

1.5 Scope of the Project

The scope of this project is the analysis and design of the Smart University ERP System. The work covers the identification of requirements, use case modelling, domain and design class modelling, interaction modelling, behavioural modelling, architectural modelling and the mapping of the class model to relational tables.

The system covers the following user groups: students, faculty members, administrative staff, examination staff and administrators, together with an external payment gateway. Activities outside this boundary, such as online learning content creation, external university accreditation and third-party recruitment services, are not part of the system.

The functions that are inside the scope of the system are:

- Registration, login and role-based access for students, faculty members, staff and administrators.
- Student admission, course registration, course details, academic schedules and timetables.
- Attendance management, examination scheduling, marks entry, result publication and fee payment.
- Assignment management, study materials, student notifications and academic progress tracking.
- Department, course, faculty and student management, along with report generation.

The following are outside the scope of the system:

- External university accreditation and government approval procedures.
- Physical classroom management, campus construction and infrastructure maintenance.
- External recruitment processes and company placement procedures.
- The internal working of the external payment gateway.

CHAPTER 2 – REQUIREMENT ANALYSIS

2.1 Functional Requirements

The functional requirements describe the services that the system must provide. They are listed in Table 2.1.

Table 2.1: Functional Requirements

ID	Requirement	Description
FR1	Student registration and login	The system shall allow students to register and log in to the portal.
FR2	Course registration	The system shall allow students to register for available courses.
FR3	Course details and timetable	The system shall display course details, class schedules and timetables.
FR4	Attendance management	The system shall allow faculty members to record and students to view attendance.
FR5	Fee Payment	The system shall process student fee payments through an external payment gateway and verify the result.
FR6	Fee status and receipts	The system shall allow students to view fee status and download payment receipts.
FR7	Examination management	The system shall allow examination staff to schedule examinations and manage examination details.
FR8	Marks and results	The system shall allow faculty members and examination staff to enter and update marks, and students to view examination results.
FR9	Faculty and course management	The system shall allow administrators to manage faculty members, courses and academic schedules.
FR10	Timetable management	The system shall allow administrators to create and update class timetables.
FR11	Assignment and study materials	The system shall allow faculty members to upload assignments and study materials for students.
FR12	Notifications	The system shall allow the university to send academic announcements and important notifications to students and faculty.
FR13	Administration	The system shall allow administrators to manage users, departments, courses and system settings.
FR14	Reports	The system shall allow administrators to generate attendance, examination, fee payment and academic performance reports.

The functional requirements are grouped by the user who needs them:

- Student requirements (FR1–FR8): Registration, course registration, timetable viewing, attendance, fee payment and examination results.
- Faculty requirements (FR4, FR8, FR9, FR11): Attendance recording, marks entry, course management and study material uploads.
- Administrator requirements (FR9, FR10, FR12–FR14): Faculty and course management, timetable management, notifications, administration and report generation.

Each functional requirement is realised by one or more of the use cases listed in Table 2.4, which makes it possible to check that no requirement is left without a use case.

2.2 Non-Functional Requirements

The non-functional requirements describe the qualities that the system must have. They are listed in Table 2.2.

Table 2.2: Non-Functional Requirements

ID	Category	Description
NFR1	Performance	The system shall remain responsive when multiple students, faculty members and administrators access it simultaneously.
NFR2	Security	Student and academic data shall be protected, and access shall depend on the role of the user.
NFR3	Extensibility	The design shall allow new courses, departments and academic services to be added without redesign.
NFR4	Data integrity	Student records, attendance, examination marks and fee payment details shall remain accurate and consistent.
NFR5	Availability	Student registration, attendance and academic services shall be available whenever the university requires them.
NFR6	Usability	The student and faculty portals shall be simple enough to use without extensive assistance.
NFR7	Maintainability	The system shall be built using modular components so that a change in one module has limited impact on others.

The design decisions that address these non-functional requirements are:

- Performance and availability: The work is divided among separate modules, so that a heavy load on one module, such as examination management, does not stop other services.
- Security: Students, faculty members and administrators log in with role-based access, and payment details are handled by the external payment gateway.
- Data integrity: The system validates student records, attendance, examination marks and fee payment details before saving them, ensuring that the data remain accurate and consistent.
- Extensibility and maintainability Students, courses, departments and faculty members are modelled as separate classes, so new courses or departments can be added without major changes to the existing code.

2.3 Actors and Their Roles

An actor is a person or an external system that interacts with the system. Five human actors and one external system have been identified. They are listed in Table 2.3.

Table 2.3: Actors and Their Roles

Actor	Type	Role
Student	Primary	Registers for courses, views timetables, checks attendance, pays fees and views examination results.
Faculty	Primary	Manages courses, records attendance, uploads assignments and enters examination marks.
Administrative Staff	Primary	Manages student admissions, student records, course registration and academic schedules.
Examination Staff	Primary	Manages examination schedules, processes marks and publishes results.
Administrator	Primary	Manages users, departments, courses, system settings and reports.
Payment Gateway	External system	Processes online fee payments and returns the payment status to the system.

The actors are divided into two groups. The primary actors start a use case to achieve their goals: the students want to access academic services, while the faculty and administrative staff want to manage university activities. The Payment Gateway is a supporting actor because it processes payment requests from the system and returns the payment status.

Student, Faculty, Administrative Staff and Examination Staff are kept as separate actors because each group is allowed to perform different operations, ensuring that access to the system depends on the user's role.

2.4 Use Case Identification

Each function that an actor needs from the system is identified as a use case. Twenty-seven use cases were identified; they are listed in Table 2.4 together with the actor that initiates them.

Table 2.4: Use Case Identification

ID	Use Case	Actor
UC01	Student Registration and Login	Student
UC02	View Student Details	Student
UC03	Register for Courses	Student
UC04	View Course Details (included in UC03)	Student
UC05	View Timetable (included in UC03)	Student
UC06	Make Fee Payment	Student
UC07	Process Payment (included in UC06)	Payment Gateway
UC08	View Fee Status	Student
UC09	Download Fee Receipt	Student
UC10	View Attendance	Student
UC11	View Examination Results	Student
UC12	View Assignments and Study Materials	Student
UC13	Submit Assignments	Student
UC14	Manage Courses	Faculty
UC15	Record Attendance	Faculty
UC16	Enter Examination Marks	Faculty
UC17	Upload Study Materials	Faculty
UC18	Manage Student Records	Administrative Staff
UC19	Manage Course Registration	Administrative Staff
UC20	Manage Timetables	Administrative Staff
UC21	Schedule Examinations	Examination Staff
UC22	Manage Examination Results	Examination Staff

ID	Use Case	Actor
UC23	Publish Examination Results	Examination Staff
UC24	Manage Users	Administrator
UC25	Manage Departments and Courses	Administrator
UC26	Manage System Settings	Administrator
UC27	Generate Reports	Administrator

2.5 Use Case Descriptions

The principal use cases are described below in Tables 2.5 to 2.13. They cover student registration, course registration, fee payment, attendance management, examination results, timetable management, assignment submission, course management and report generation.

Table 2.5: Use Case Description – Student Registration

Use Case ID	UC01
Use Case Name	Student Registration and Login
Actor(s)	Student
Description	The student registers on the Smart University ERP system and logs in to access academic services.
Preconditions	The student has valid registration details and access to the university portal.
Main Flow	1. The student opens the ERP portal. 2. The student enters the required registration details 3. The system validates the entered information. 4. The system creates the student account. 5. The student logs in and accesses the dashboard.
Alternative Flow	3a. Invalid details: the system displays an error message. 4a. Existing account: the system asks the student to log in instead of registering again
Postconditions	The student account is created successfully, and the student can access the ERP portal.

Table 2.6: Use Case Description – Make Fee Payment

Use Case ID	UC06
Use Case Name	Make Fee Payment

Actor(s)	Student, Payment Gateway
Description	The student pays university fees through the external payment gateway.
Preconditions	The student is logged in, and the pending fee amount is available.
Main Flow	1. The student chooses a payment method. 2. The system sends the student ID, fee details and amount to the payment gateway. 3. The gateway processes the payment and returns the payment status. 4. The system generates a payment receipt.
Alternative Flow	3a. The payment fails: the system offers Retry Payment and returns to step 1.
Postconditions	The payment is recorded against the student's fee account with its status, and a receipt is generated after successful payment.

Table 2.7: Use Case Description – Cancel Course Registration

Use Case ID	UC19
Use Case Name	Cancel Course Registration
Actor(s)	Student, Administrative Staff
Description	The student cancels an existing course registration through the Smart University ERP system.
Preconditions	The student is logged in, and an existing course registration is available for cancellation.
Main Flow	1. The student selects the course registration to cancel. 2. The system checks the course registration and cancellation rules. 3. If a fee refund is applicable, the system initiates the refund process.
Alternative Flow	3a. If cancellation is not permitted, the system displays an error message and keeps the course registration unchanged.
Postconditions	The course registration is cancelled, and the student's academic record is updated.

Table 2.8: Use Case Description – View Attendance

Use Case ID	UC10
Use Case Name	View Attendance
Actor(s)	Student

Description	The student views attendance records for registered courses through the Smart University ERP system.
Preconditions	The student is logged in, and attendance records are available.
Main Flow	1. The student opens the attendance module. 2. The system retrieves the student's attendance records. 3. The system displays course-wise attendance details. 4. The student views the attendance percentage.
Alternative Flow	If attendance records are unavailable, the system displays an appropriate message.
Postconditions	The student can view the available attendance records and attendance percentage.

Table 2.9: Use Case Description – Update Examination Results

Use Case ID	UC22
Use Case Name	Manage Examination Results
Actor(s)	Examination Staff
Description	Examination staff update examination results and manage student marks in the Smart University ERP system.
Preconditions	The staff member is logged in, authorized to manage results, and the examination records exist.
Main Flow	1. The staff member selects an examination. 2. The staff member enters or updates student marks. 3. The system validates the marks and saves the updated results.
Alternative Flow	If the marks are invalid or the update is not permitted, the system rejects the changes and displays an error message.
Postconditions	The examination results are updated and stored in the system.

Table 2.10: Use Case Description – Assign Faculty to Course

Use Case ID	UC25
Use Case Name	Assign Faculty to Course
Actor(s)	Administrator
Description	The administrator assigns a faculty member to a course in the Smart University ERP system.
Preconditions	The course exists, the faculty member is registered, and the administrator is logged in.

Main Flow	1. The administrator selects a course. 2. The administrator selects a faculty member. 3. The system validates the assignment and links the faculty member to the course.
Alternative Flow	If the faculty member is unavailable or the assignment is invalid, the system displays an error message and asks the administrator to select another faculty member.
Postconditions	The faculty member is assigned to the course, and the assignment details are saved in the system.

Table 2.11: Use Case Description – Check Academic Results

Use Case ID	UC11
Use Case Name	View Examination Results
Actor(s)	Student
Description	The student checks the current examination results through the Smart University ERP system.
Preconditions	The student is logged in, and the examination results are available in the system.
Main Flow	1. The student selects a semester or examination. 2. The system retrieves the student's examination records. 3. The system displays subject-wise marks, grades and result status.
Alternative Flow	If the results are not available, the system displays a message asking the student to check again later.
Postconditions	The examination results are displayed to the student.

Table 2.12: Use Case Description – Manage Student Records

Use Case ID	UC18
Use Case Name	Manage Student Records
Actor(s)	Administrative Staff
Description	Administrative staff maintain student records and update academic and personal information in the Smart University ERP system.
Preconditions	The staff member is logged in and authorized to manage student records.

Main Flow	1. The staff member selects a student. 2. The staff member enters or updates student details. 3. The system validates the information and saves the changes. 4. The system displays a confirmation message.
Alternative Flow	If the entered information is invalid, the system displays an error message and asks the staff member to correct it.
Postconditions	The student record is saved with the updated information.

Table 2.13: Use Case Description – Schedule Examinations

Use Case ID	UC21
Use Case Name	Schedule Examinations
Actor(s)	Examination Staff
Description	Examination staff schedule examinations and manage examination details for students in the Smart University ERP system.
Preconditions	The staff member is logged in and authorized to manage examination schedules.
Main Flow	1. The staff member selects a course or semester. 2. The staff member enters the examination date, time and venue. 3. The system checks for scheduling conflicts. 4. The system saves and publishes the examination schedule.
Alternative Flow	If a scheduling conflict is detected, the system displays an error message and asks the staff member to select another date or time.
Postconditions	The examination schedule is saved and made available to students and faculty.

2.6 Event List

An event is something that happens, outside or inside the system, to which the system must respond. The events identified for the Smart University ERP System are given below.

- E1: Student registers for an account.
- E2: Student logs in to the ERP system.
- E3: Student registers for courses.
- E4: Student views course details.
- E5: Student views the timetable.
- E6: Student makes a fee payment.

- E7: Payment Gateway returns the payment status.
- E8: Student requests a fee receipt.
- E9: Faculty records student attendance.
- E10: Student views attendance details.
- E11: Faculty enters examination marks.
- E12: Examination staff schedule examinations.
- E13: Examination results are published.
- E14: Student views examination results.
- E15: Faculty uploads study materials.
- E16: Administrator updates university records.
- E17: Administrator requests an academic, attendance or fee report.

2.7 Event Table

Table 2.14 relates each event to its trigger, its source, the use case that handles it and the response of the system.

Table 2.14: Event Table

No.	Event	Trigger	Source	Use Case	Response
E1	Student registers for an account	Student submits registration details	Student	UC01	Student account is created
E2	Student logs in to the ERP system	Student enters login credentials	Student	UC01	Login is verified and dashboard is displayed
E3	Student registers for courses	Student selects courses	Student	UC03	Course registration is saved
E4	Student views course details	Student requests course information	Student	UC04	Course details are displayed
E5	Student views timetable	Student requests timetable	Student	UC05	Student timetable is displayed
E6	Student makes a fee payment	Student submits payment details	Student	UC06	Payment request is sent to the payment gateway
E7	Payment gateway returns payment status	Payment processing is completed	Payment Gateway	UC07	Payment status is updated

No.	Event	Trigger	Source	Use Case	Response
E8	Student requests fee receipt	Student selects receipt option	Student	UC09	Fee receipt is generated and displayed
E9	Faculty records attendance	Faculty enters attendance details	Faculty	UC15	Attendance records are saved
E10	Student views attendance	Student requests attendance details	Student	UC10	Course-wise attendance is displayed
E11	Faculty enters examination marks	Faculty submits marks	Faculty	UC16	Marks are validated and saved
E12	Examination staff schedules examinations	Staff enters exam details	Examination Staff	UC21	Examination schedule is saved
E13	Examination results are published	Examination staff publishes results	Examination Staff	UC23	Results become available to students
E14	Student views examination results	Student selects an examination	Student	UC11	Marks, grades and result status are displayed
E15	Faculty uploads study materials	Faculty uploads files	Faculty	UC17	Study materials are uploaded and made available
E16	Administrator updates university records	Administrator submits updated details	Administrator	UC24 ,UC 25	User, department or course records are updated
E17	Administrator requests an academic report	Administrator selects report criteria	Administrator	UC27	The requested report is generated

CHAPTER 3 – USE CASE AND DOMAIN MODELLING

3.1 Use Case Modelling

3.1.1 Use Case Diagram

Figure 3.1 shows the use case diagram of the Smart University ERP.

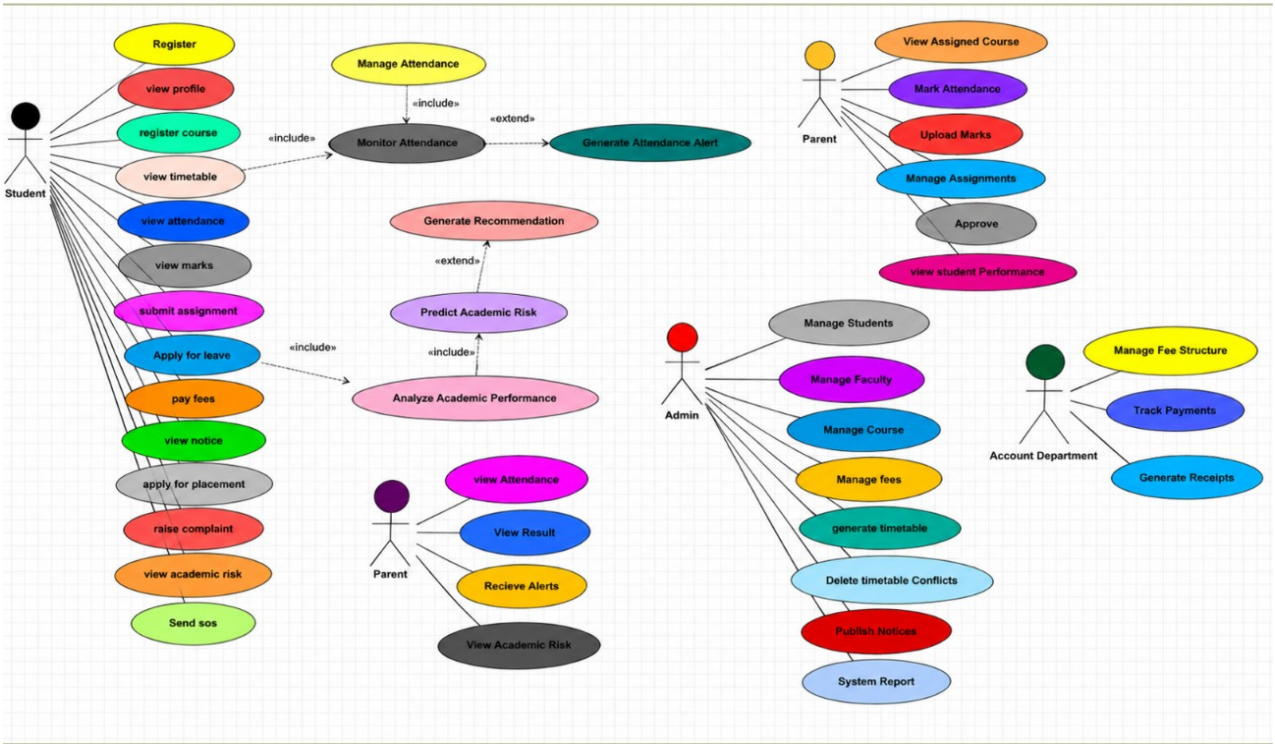


Figure 3.1: Use Case Diagram

3.1.2 Description and Terminology

A use case diagram shows the functions of the Smart University ERP (Enterprise Resource Planning) System from the point of view of its users. The main elements are the following.

- Actor: A role played by a person who interacts with the system, such as Admin, Faculty, Student, and Parent.
- Use case: A function performed by the system, such as student registration, attendance management, fee payment, examination management, and result viewing.
- System boundary: The rectangle that separates the use cases inside the Smart University ERP System from the external actors.

- Association: The line between an actor and a use case that shows the actor's participation in a function, such as a student viewing attendance.
- Include: A relationship in which one use case always uses the behaviour of another use case. For example, Fee Payment includes Payment Verification.
- Extend: A relationship in which an optional use case adds behaviour to a base use case under a condition. For example, Forgot Password extends Login when a user forgets their password.

3.1.3 Project-Specific Explanation

The diagram contains six actors: Student, Faculty, Administrator, Parent, Accountant and the Notification System. The Student is associated with the use cases for student registration, login, viewing attendance, checking examination results, viewing the timetable, submitting assignments, paying fees and viewing academic details.

Student Registration includes Enter Student Details, because a student must provide the required information during registration. Fee Payment includes Payment Verification. Generate Reports extends the View Academic Details use case when a detailed academic report is required. The Notification System is an external actor associated with sending notifications about attendance, examinations, fees and announcements.

Faculty manage student attendance, upload study materials, assign homework, conduct examinations and update marks. The Administrator manages student and faculty records, courses, departments, timetables and system settings and generates reports. The Accountant manages fee records, verifies payments and maintains financial reports. Parents view their children's attendance, examination results, fee details and academic progress.

The use case diagram also helps to check the completeness of the requirements. Every actor has at least one use case, every use case is associated with an actor, and every function listed in Table 2.1 appears in the diagram. The include and extend relationships are used only where the behaviour is shared or optional, so that the diagram remains easy to read.

- Student: Eight use cases for registration, login, attendance, results, timetable, assignments, fee payment and academic details.
- Faculty: Four use cases for attendance, study materials, homework and marks.
- Administrator: Five use cases for student records, faculty records, courses, system settings and reports.

- Parent: Four use cases for attendance, results, fee details and academic progress.
- Accountant: Four use cases for fee records, payment verification, financial records and fee reports.
- Notification System: One use case for sending notifications about attendance, examinations, fees and announcements.

3.2 Domain Modelling

3.2.1 Domain Class Diagram

Figure 3.2 shows the domain classes of the system and the associations between them.

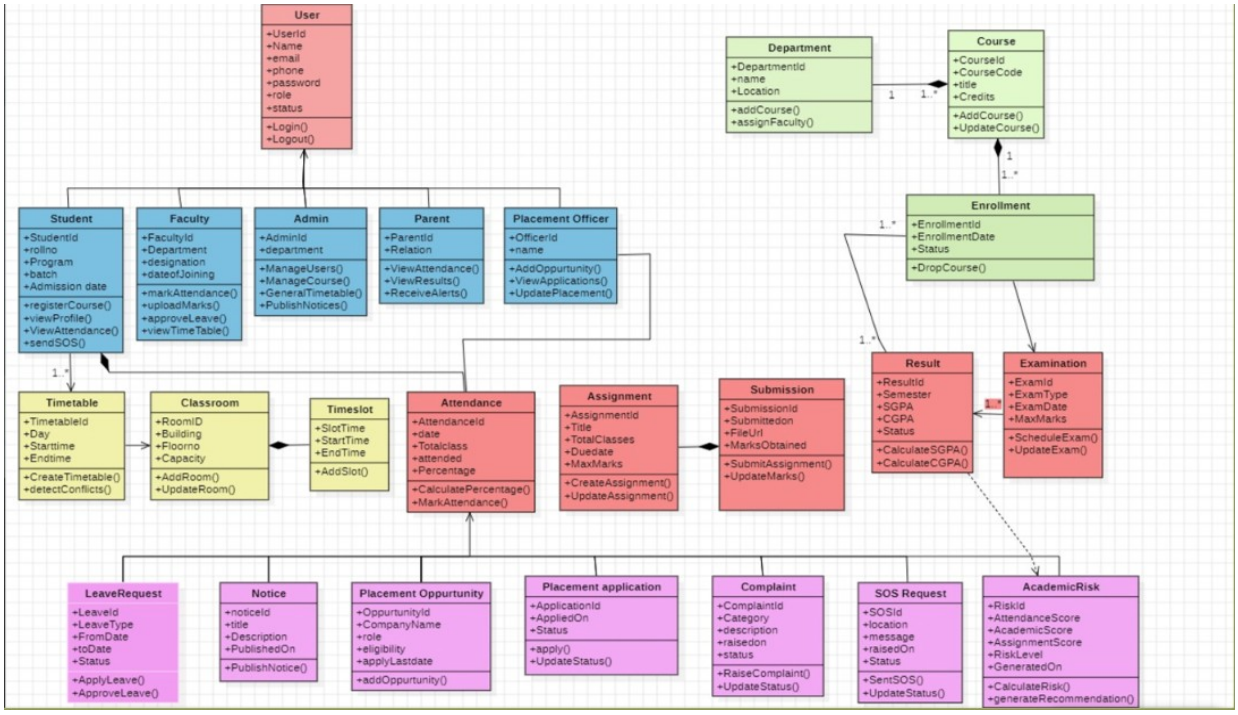


Figure 3.2: Domain Class Diagram

3.2.2 Description and Terminology

A domain model describes the concepts of the Smart University ERP System and the relationships between them. It is drawn as a UML class diagram. The terms used are as follows.

- Class: A description of a set of objects that have the same attributes and operations, such as Student, Faculty, Course and Department.
- Attribute: A property of a class, such as studentId, studentName, courseName or attendanceStatus.

- Association: A relationship between two classes, such as Student and Course or Faculty and Department.
- Multiplicity: The number of objects of one class that can be linked to one object of another class, for example 0..* or 1.
- Generalization: An inheritance relationship in which a subclass specializes a superclass, such as Admin, Faculty and Student inheriting common properties from a User class.

3.2.3 Project-Specific Explanation

The central classes are Student, Faculty, Course, Department, Attendance, Examination, Result, Fee Payment and Administrator. A Student can enroll in many Courses, and each Course can have many Students. A Student can have many Attendance records and Examination Results. A Faculty member can teach many Courses, and each Course is managed by a Department. A Student can make many Fee Payments, and each payment belongs to one Student.

User is a superclass with three subclasses: Student, Faculty and Administrator. This inheritance lets all three user types share common attributes and operations, such as login and logout, while each subclass adds its own specific attributes and operations.

The responsibility of each domain class is as follows:

- Student: Holds student ID, name, contact details and academic information, and manages registration, course enrollment and academic detail viewing.
- Course: Records course ID, course name, credits and syllabus, and manages course information.
- Faculty: Holds faculty ID, name, contact details and designation, and manages attendance, assignments and marks.
- Department: Holds department ID, department name and details, and manages courses and faculty.
- Attendance: Records attendance date, status and student details, and tracks student attendance.
- Examination: Holds exam ID, subject, date and maximum marks, and manages examination schedules.

- Result: Records marks, grades and result status, and provides examination results to students.
- Fee Payment: Records payment ID, amount, mode and payment status, and handles fee payment and verification.
- Administrator: Holds administrator login details and role permissions, and manages users, courses, system settings and reports.

CHAPTER 4 – INTERACTION MODELLING

Interaction models show how objects exchange messages to carry out a use case in the Smart University ERP System. This chapter presents the system sequence, high-level sequence, detailed sequence and communication diagrams for student registration, attendance management, fee payment and examination result scenarios.

4.1 System Sequence Diagram

A system sequence diagram treats the Smart University ERP System as a single black box and shows only the messages between the actor and the system. For the Student Registration use case, the student enters the registration details, submits the required information and provides login credentials; the system replies with registration status, validates the details, creates the student account and displays the registration confirmation.

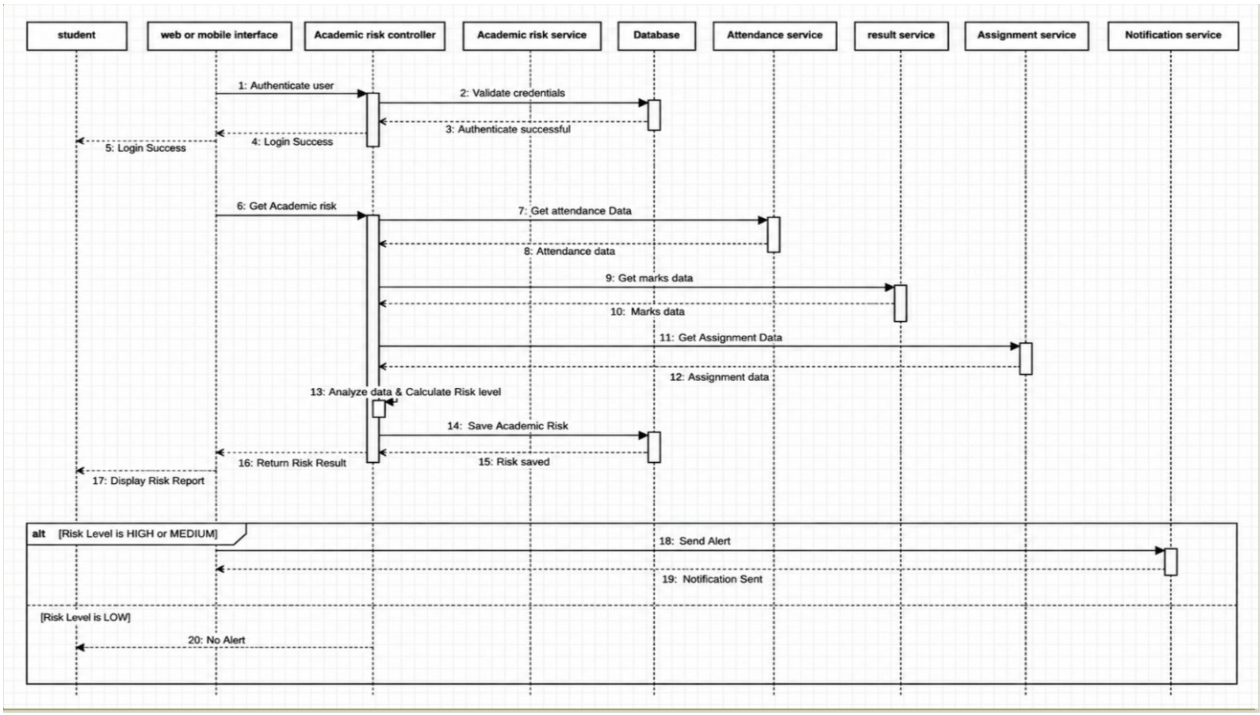


Figure 4.1: System Sequence Diagram

CHAPTER 5 – DESIGN CLASS MODELLING

5.1 Design Class Diagram

The design class diagram refines the domain model by adding visibility, data types and operations to each class, so that it can be converted directly into code. Figure 5.1 shows the design class diagram of the system.

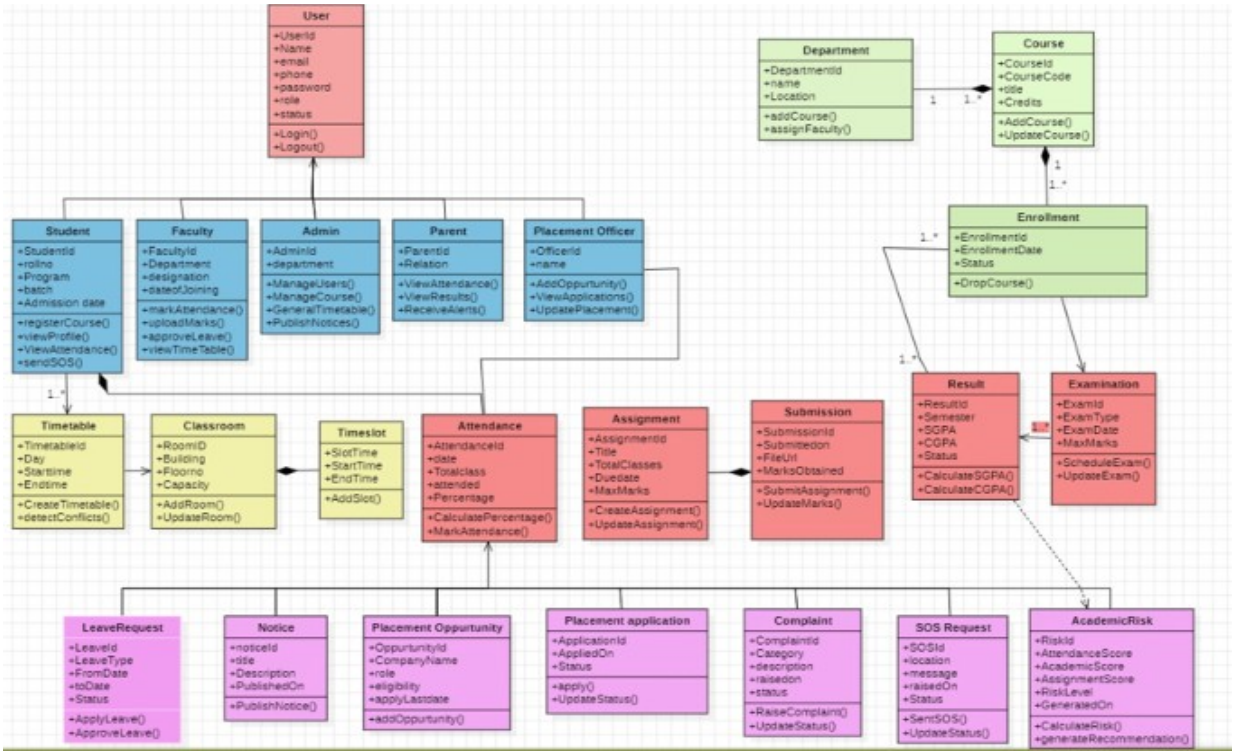


Figure 5.1: Design Class Diagram

5.2 Classes, Attributes and Operations

Table 5.1 lists the classes of the design class diagram with their attributes and operations. All attributes are private, which applies the principle of encapsulation: they can be read or changed only through the public operations of the class.

Table 5.1: Classes, Attributes and Operations

Class	Attributes	Operations
Student	studentId, rollNo, name, email, mobileNo	registerStudent(), updateProfile(), viewAttendance(), viewResults(), updateStatus()

Class	Attributes	Operations
Faculty	facultyId, name, email, department, designation	updateFaculty(), markAttendance(), uploadMarks(), viewCourses()
Course	courseId, courseName, courseCode, credits	addCourse(), updateCourse(), viewCourseDetails()
Attendance	attendanceId, date, status, remarks	recordAttendance(), updateAttendance(), viewAttendance()
Exam	examId, examName, examDate, totalMarks, examType	scheduleExam(), updateExam(), viewExamDetails()
Payment	paymentId, amount, mode, status	processPayment(), refundPayment()
Results	resultId, marks, grade, status	calculateResult(), publishResult(), viewResult()
User	userId, name, email, password, role	login(), logout()
Fees	feeId, amount, dueDate, status	viewFeeStatus(), generateReceipt(), updateFeeStatus()
Library	bookId, bookTitle, author, availabilityStatus	issueBook(), returnBook(), searchBook()
Admin	adminId, employeeId, role	manageUsers(), manageCourses(), generateReports()

The status attributes are typed by enumerations: Student.status uses StudentStatus, Attendance.status uses AttendanceStatus, and Fees.status uses FeeStatus.

The following points explain how the classes support the principles of object-oriented design:

- Encapsulation The attributes of every class are private, so the state of an object can be changed only by its own operations, such as updateStatus() in Student.
- Abstraction: Each class shows only the operations that other classes need, for example, recordAttendance() in Attendance and processPayment() in Payment.
- Inheritance: Student, Faculty, and Admin reuse common attributes and operations from the User superclass, such as login() and logout().
- Association: Student, Course, and Faculty are linked by associations, so student course registrations, faculty assignments, and attendance records can be managed consistently.

5.3 Relationships and Multiplicity

The relationships between the classes, with their multiplicities, are given in Table 5.2.

Table 5.2: Relationships and Multiplicity

Classes	Relationship	Multiplicity	Meaning
Student – Course	Association	1 to 0..*	A student can register for multiple courses.
Course – Faculty	Association	0..* to 1	A faculty member can teach multiple courses.
Student – Attendance	Association	1 to 0..*	A student can have multiple attendance records.
Faculty – Attendance	Association	1 to 0..*	A faculty member can record multiple attendance entries.
Course – Exam	Association	1 to 0..*	A course can have multiple examinations.
Student – Results	Association	1 to 0..*	A student can have multiple examination results
Student – Fees	Association	1 to 0..*	A student can have multiple fee records.
Fees – Payment	Association	1 to 0..1	A fee record can have at most one payment record.
Student – Library	Association	1 to 0..*	A student can borrow multiple library books over time.

5.4 Interfaces, Dependencies and Generalization

Generalization: User is the superclass of Student, Faculty, and Admin. The subclasses inherit userId, name, email, password, login() and logout(), and add student-specific details, faculty details, or administrative roles with their own operations.

Dependencies: The Student class depends on Course, Attendance, Results, Fees, and Library, because its operations register for courses, view attendance, check results, pay fees, and access library services. Attendance depends on Student and Faculty, because faculty members record attendance for registered students. Results depends on Exam, because examination marks are used to calculate and publish student results.

Interfaces: The Fees Management component is the part of the system that communicates with the external Payment Gateway. This isolates the gateway so that it can be replaced without changing the Student or Fees classes. The Boundary–Control–Entity layering also separates the user interface screens, the controllers that process requests, and the entities that store university data.

The design follows these principles:

- High cohesion: Each class has one clear purpose, such as Student managing student details, Attendance managing attendance records, and Fees managing fee payments.
- Low coupling: Classes interact through associations and public operations, so changes in one module do not unnecessarily affect other modules.
- Extensibility: New modules, such as Transport Management or Hostel Management, can be added to the Smart University ERP system without making major changes to existing classes.

CHAPTER 6 – BEHAVIOURAL MODELLING

6.1 Activity Diagram

An activity diagram shows the flow of work in a process, including decisions and loops. Figure 6.1 shows the Student Registration workflow in the Smart University ERP system. The student logs in, enters student details, selects a course, and the system checks course availability. If the course is unavailable, the flow returns to Course Selection. Otherwise, the student submits the registration form, the system creates a student record, and the registration fee is processed. If the payment fails, the student goes to Retry Payment and returns to Process Fee Payment; if it succeeds, the student registration is confirmed. These branches and retry loops help describe the workflow clearly alongside the sequence diagrams.

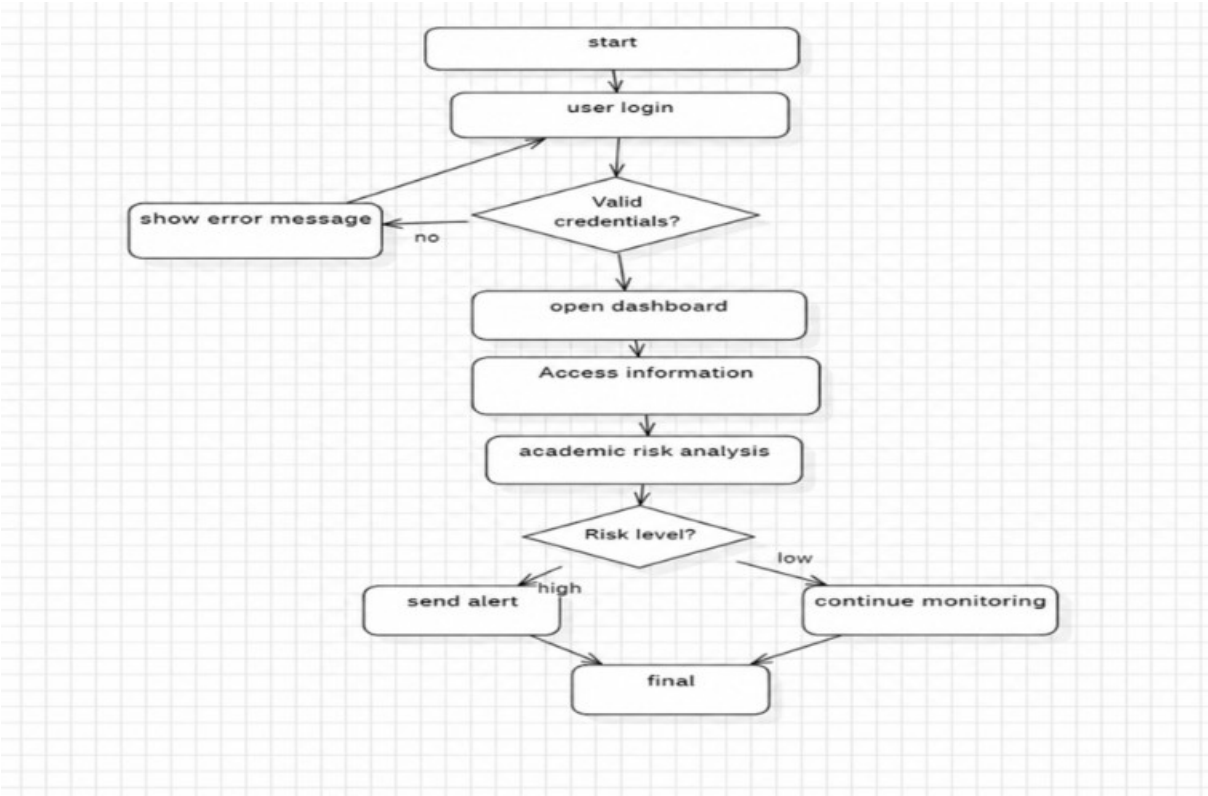


Figure 6.1: Activity Diagram

Table 6.1: Actions and Decisions in the Student Registration Activity

No.	Action or Decision	Description
1	Login / Register	The student logs in, or registers if there is no account yet.
2	Enter Student Details	The student enters personal and academic details.
3	Verify Student Details	The system checks whether the entered details are valid.
4	Check Course Availability	The system checks whether the selected course is available for registration.
5	Decision: course available?	If the course is unavailable, the flow returns to Course Selection; otherwise, it continues.
6	Submit Registration	The student submits the registration form for approval.
7	Create Student Record	The system creates a student record in the database.
8	Process Fee Payment	The system processes the registration fee payment through the payment gateway.
9	Decision: payment successful?	If the payment fails, the flow goes to Retry Payment and returns to Process Fee Payment.
10	Registration Confirmed	The student registration is confirmed after the payment succeeds.

6.2 State Machine Diagram

A state machine diagram shows the states that an object passes through during its life and the events that cause the changes. Figure 6.2 shows the lifecycle of a Student in the Smart University ERP system. A student starts as Registered and becomes Active when the registration is approved. The student progresses to Enrolled when course registration is completed and becomes Exam Eligible when the attendance and examination requirements are satisfied. The student becomes Graduated when all academic requirements are completed, which is the final state. A Registered student can instead become Cancelled when the admission is cancelled, which is a second final state.

The diagram defines the values of Student.status and guides the behaviour of the updateStatus() operation, which must accept only the transitions shown.

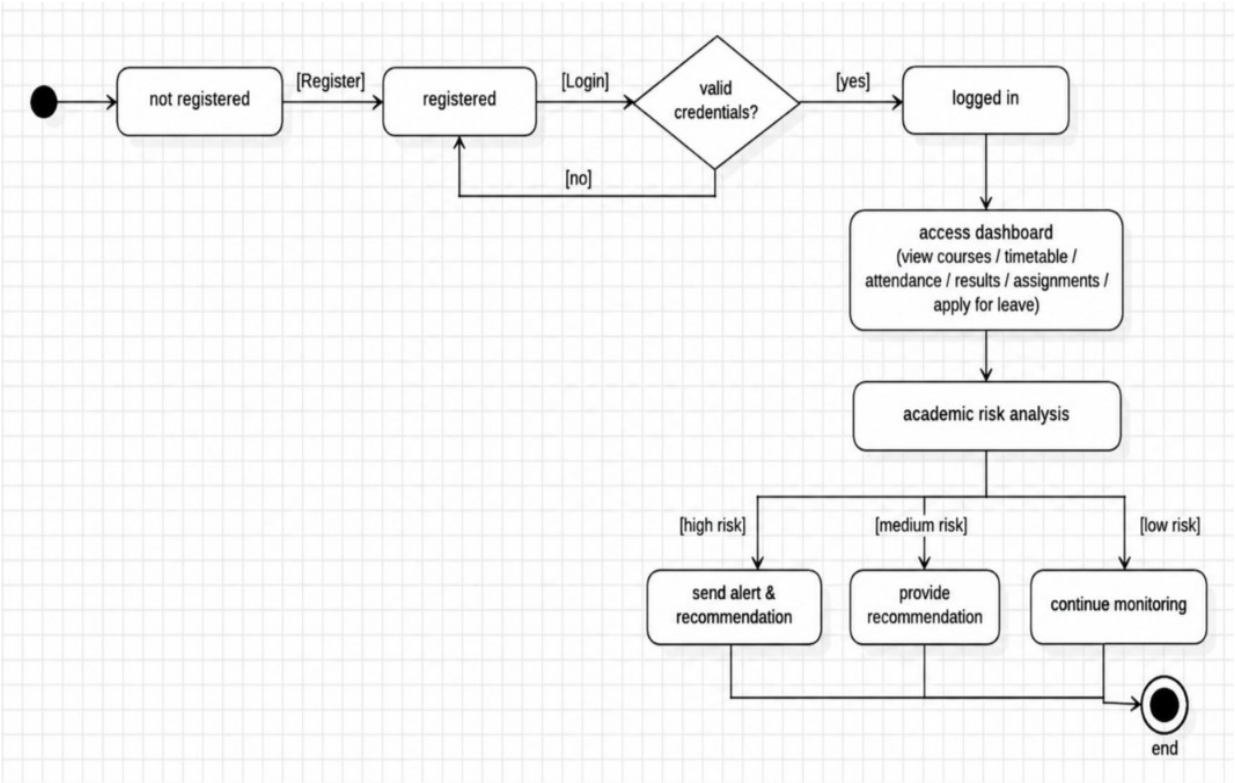


Figure 6.2: State Machine Diagram – Student Lifecycle

Table 6.2: Transitions of the Student Lifecycle

From State	Event or Condition	To State	Remark
Start	Student is registered	Registered	Initial state
Registered	Registration is approved	Active	Account activated
Active	Course registration is completed	Enrolled	Course registered
Enrolled	Attendance and examination requirements are satisfied	Exam Eligible	Eligible for examinations
Exam Eligible	All academic requirements are completed	Graduated	Final state
Registered	Admission is cancelled	Cancelled	Final state

Because no transition leaves the Graduated or Cancelled state in this model, a completed or cancelled student record cannot be changed through the normal workflow. The operation updateStatus() therefore accepts only the transitions listed in Table 6.2, helping protect the student status from incorrect changes.

CHAPTER 7 – ARCHITECTURAL MODELLING

7.1 Package Diagram

A package diagram groups the classes into modules and shows the dependencies between them. Figure 7.1 shows eight packages: Student Management, Faculty Management, Course Management, Attendance Management, Examination Management, Results Management, Fees Management, and Library Management.

Student Management depends on Course Management, Fees Management, and Notification Management. Faculty Management depends on Attendance Management and Examination Management. Examination Management depends on Results Management, and Library Management depends on Student Management. The packages also use shared services such as Notification Management, and the diagram avoids cyclic dependencies, which keeps the modules independent and easy to maintain and modify.

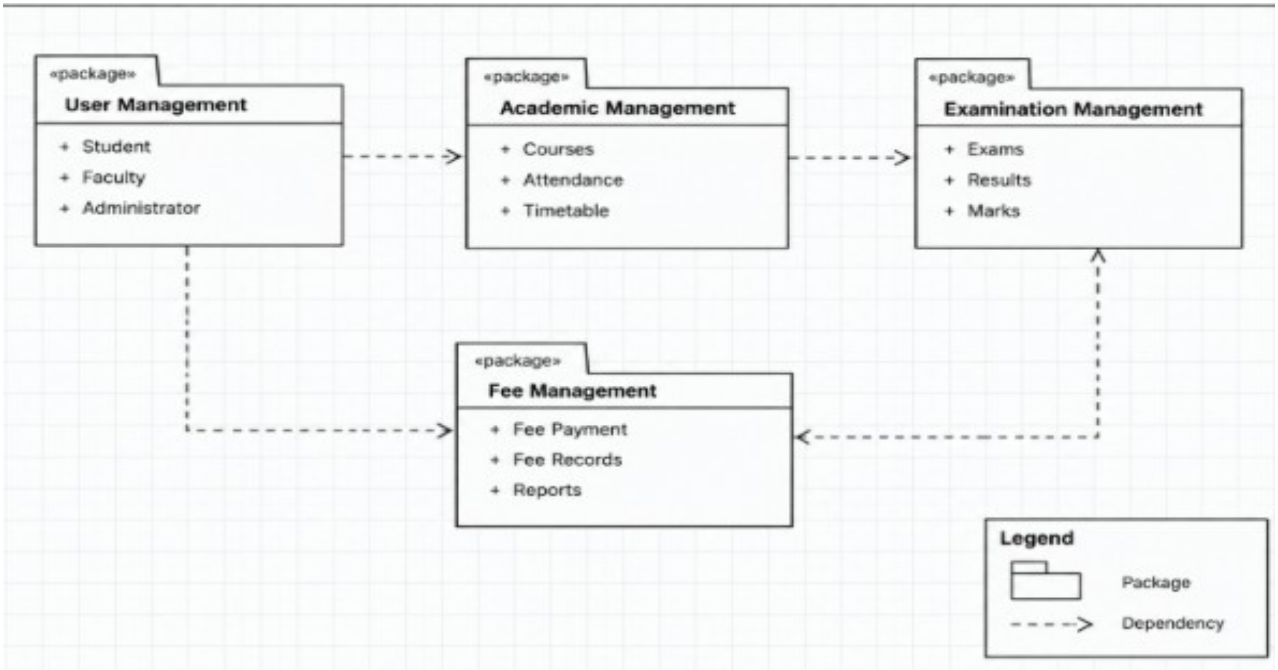


Figure 7.1: Package Diagram

The dependencies that are described for the packages are listed in Table 7.1.

Table 7.1: Package Dependencies

Package	Depends on	Reason
Student Management	Course Management	Students register for courses offered by the university.

Package	Depends on	Reason
Student Management	Fees Management	Student fee details are maintained for each registered student.
Student Management	Notification Management	Students receive important university announcements and updates.
Faculty Management	Attendance Management	Faculty members record and manage student attendance.
Faculty Management	Examination Management	Faculty members prepare and manage examinations.
Examination Management	Results Management	Examination marks are used to prepare and publish student results.

Keeping the dependencies in one direction and avoiding cycles means that each package can be tested and changed independently, and the packages can be developed in the order of their dependencies.

7.2 Component Diagram

A component diagram shows the software components of the system and the interfaces through which they communicate. Figure 7.2 shows the components of the Smart University ERP System.

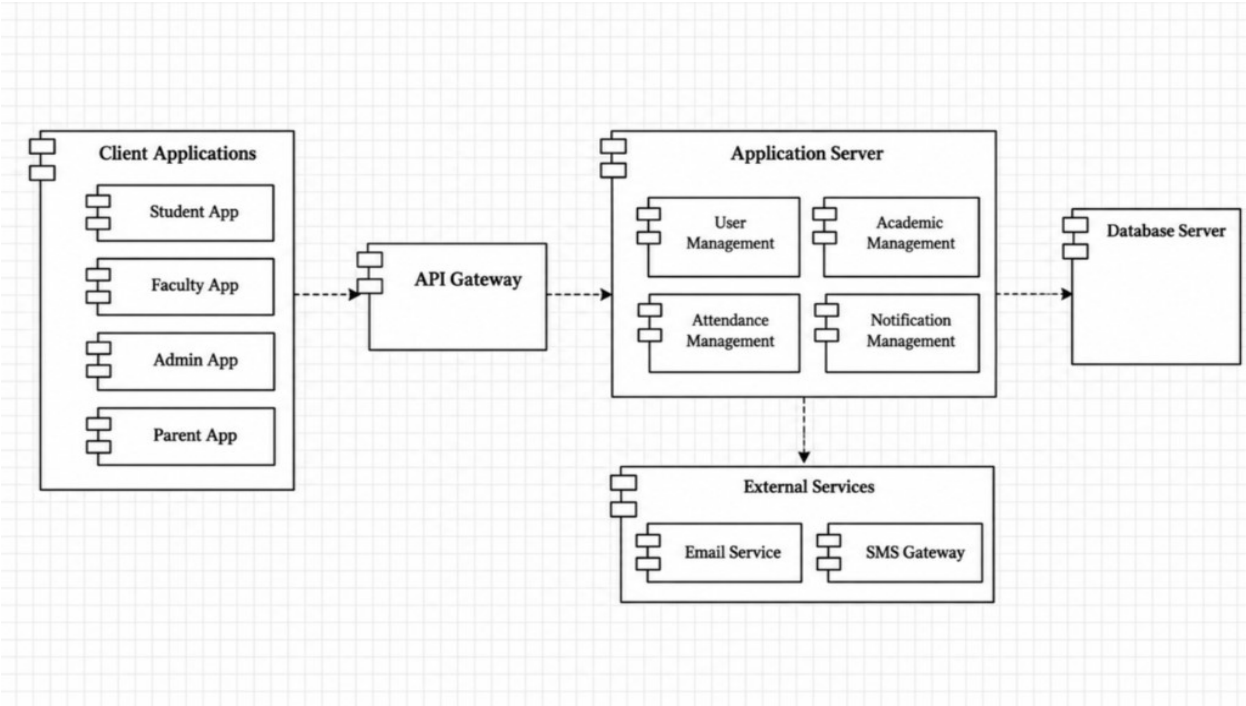


Figure 7.2: Component Diagram

The responsibilities of the components are listed in Table 7.2.

Table 7.2: Components and Their Responsibilities

Component	Responsibility
Student Portal	The screen through which students access the Smart University ERP system.
Student Management Service	Creates, updates, and manages student records and registration details.
Faculty Management Service	Maintains faculty information, department details, and faculty records.
Attendance Service	Records student attendance and generates attendance reports.
Examination Service	Manages examination schedules, registrations, and exam details.
Results Service	Stores, updates, and publishes student examination results.
Fees Management Service	Manages fee details, payment records, and outstanding balances.
Library Service	Maintains library books and manages book issue and return records.
Notification Service	Sends notifications to students and faculty through e-mail or SMS.
Payment Gateway	External interface used to process online university fee payments.

7.3 Deployment Diagram

A deployment diagram shows how the software artifacts are placed on hardware nodes. In Figure 7.3, the Student Device connects to the Web Server, which hosts the Smart University ERP Portal. The Web Server connects to the Application Server, which hosts the student management, faculty management, attendance, examination, results, fees, and library services. The Application Server connects to the Database Server, which stores the University ERP Database, and to the Payment Gateway Server for online fee payments and the Notification Server for e-mail and SMS notifications.

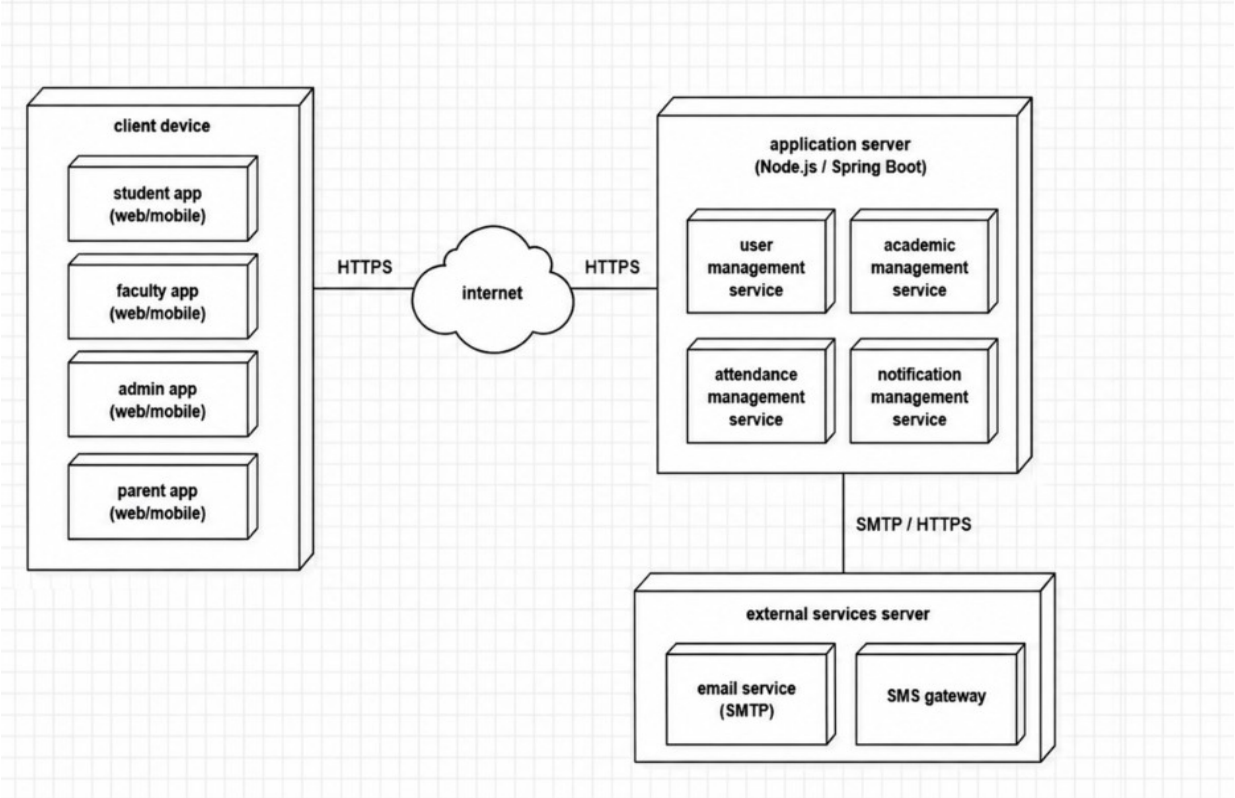


Figure 7.3: Deployment Diagram

The nodes and the artifacts that are deployed on them are listed in Table 7.3.

Table 7.3: Nodes and Deployed Artifacts

Node	Artifacts	Connected to
Student Device	Student Portal client	Web Server
Web Server	Smart University ERP Web Portal	Application Server

Node	Artifacts	Connected to
Application Server	Student Management, Faculty Management, Attendance, Examination, Results, Fees and Library Services	Database Server, Payment Gateway Server, Notification Server
Database Server	University ERP Database	Application Server
Payment Gateway Server	Online Fee Payment Gateway	Application Server
Notification Server	Email and SMS Notification Services	Application Server

Separating the web server, application server, and database server has three benefits: the database is not directly accessible from student devices, each server can be given more capacity when the workload increases, and payment and notification services can be replaced without changing the main ERP application.

CHAPTER 8 – DATABASE DESIGN

8.1 Mapping Class Diagram to Relational Database

The class diagram is mapped to relational tables by the following rules.

- Each class becomes a table, and each attribute becomes a column.
- The identifier attribute of each class (for example, studentId) becomes the primary key of its table.
- An association with multiplicity 1 to 0..* is implemented by a foreign key in the table on the many side, such as STUDENT and ATTENDANCE.
- An association with multiplicity 1 to 0..1 is implemented by a foreign key with a unique constraint.
- A generalization is implemented by a table for the superclass and one table for each subclass, which shares the primary key of the superclass.

Student and other classes may require surrogate keys, such as studentId, facultyId, and courseId, to uniquely identify records in the database. The operations of the classes are not stored in the database; they are implemented in the application code.

The tables are designed to avoid repeated data. Each table describes one kind of object, every column depends on the key of its table, and a fact such as the name of a department is stored once in DEPARTMENT and referred to by departmentId. This corresponds to the third normal form (3NF) and helps maintain data consistency in the Smart University ERP system.

8.2 Database Tables

The tables obtained from the class model are given in Tables 8.1 to 8.10.

Table 8.1: STUDENT

Column	Data Type	Constraint
studentId	INT	Primary key
rollNo	VARCHAR(20)	Unique, not null
name	VARCHAR(100)	Not null
email	VARCHAR(100)	Unique, not null
address	VARCHAR(200)	
mobileNo	VARCHAR(15)	Not null

Table 8.2: DEPARTMENT

Column	Data Type	Constraint
departmentId	INT	Primary key
name	VARCHAR(100)	Not null
code	VARCHAR(10)	Unique, not null
contactNo	VARCHAR(15)	

Table 8.3: COURSE

Column	Data Type	Constraint
courseId	INT	Primary key
courseCode	VARCHAR(10)	Unique, not null
courseName	VARCHAR(100)	Not null
departmentId	INT	Foreign key to DEPARTMENT

Table 8.4: FACULTY

Column	Data Type	Constraint
facultyId	INT	Primary key
name	VARCHAR(100)	Not null
email	VARCHAR(100)	Unique, not null
password	VARCHAR(255)	Not null (stored as a hash)
designation	VARCHAR(30)	Not null

Table 8.5: RESULTS

Column	Data Type	Constraint
resultId	INT	Primary key
marks	INT	Not null
studentId	INT	Foreign key to STUDENT
examId	INT	Foreign key to EXAM

Table 8.6: ADMIN

Column	Data Type	Constraint
adminId	INT	Primary key
employeeId	INT	Unique, not null
role	VARCHAR(30)	Not null

Table 8.7: ATTENDANCE

Column	Data Type	Constraint
attendanceId	INT	Primary key
attendanceDate	DATE	Not null
status	VARCHAR(20)	Not null(AttendanceStatus)
studentId	INT	Foreign key to STUDENT
courseId	INT	Foreign key to COURSE
facultyId	INT	Foreign key to FACULTY
semester	INT	Not null
section	VARCHAR(10)	Not null
remarks	VARCHAR(100)	Nullable
recordedAt	DATETIME	Not null

Table 8.8: EXAM

Column	Data Type	Constraint
examId	INT	Primary key
examName	VARCHAR(50)	Not null
examDate	DATE	Not null
totalMarks	INT	Not null
examType	VARCHAR(20)	Not null(ExamType)
courseId	INT	Foreign key to COURSE

Table 8.9: FEES

Column	Data Type	Constraint
feeId	INT	Primary key
amount	DECIMAL(10,2)	Not null
dueDate	DATE	Not null
status	VARCHAR(20)	Not null (FeeStatus)
studentId	INT	Foreign key to STUDENT

Table 8.10: LIBRARY

Column	Data Type	Constraint
bookId	INT	Primary key
bookTitle	VARCHAR(100)	Not null
author	VARCHAR(100)	Not null
availabilityStatus	VARCHAR(20)	Not null
studentId	INT	Foreign key to STUDENT (if assigned or issued)

8.3 Primary Keys and Foreign Keys

Table 8.11 summarises the primary keys and foreign keys of the database.

Table 8.11: Primary Keys and Foreign Keys

Table	Primary Key	Foreign Key	References
STUDENT	studentId	–	–

Table	Primary Key	Foreign Key	References
FACULTY	facultyId	–	–
COURSE	courseId	–	–
ATTENDANCE	attendanceId	studentId,courseId,facultyId	STUDENT,COURSE,FACULTY
EXAM	examId	courseId	COURSE(courseId)
RESULTS	resultId	studentId,examId	STUDENT(studentId),EXAM(examId)
FEES	feeId	studentId	STUDENT(studentId)
PAYMENT	paymentId	feeId	FEES(feeId)
LIBRARY	bookId	–	–
BOOK_ISSUE	issueId	studentId,bookId	STUDENT(studentId),LIBRARY(bookId)

The following rules keep the data correct:

- Referential integrity: a foreign key must refer to an existing row, so attendance records, examination results, and fee records cannot be saved for a student or faculty member who does not exist.
- One payment record for each fee transaction: The student fee payment reference must be unique where only one payment record is allowed per transaction, preventing duplicate payment records.
- Optional faculty or course assignment: Faculty or course IDs may be empty where the design permits an assignment to be made later.
- Unique values: Student registration numbers, faculty IDs, university e-mail addresses, and course codes cannot be repeated.
- Valid attendance records: Attendance must be recorded only for registered students and valid courses, preventing duplicate attendance entries for the same student, course, and class session.
- Valid examination results: Results must be associated with an existing student and examination, ensuring that marks are stored for the correct student and exam.
- Valid library records: Book issue and return records must refer to existing students and library books, keeping library transactions accurate.

8.4 CRUD Matrix

The CRUD matrix in Table 8.12 shows which use cases Create (C), Read (R), Update (U), or Delete (D) the data of each module in the Smart University ERP system. Student, faculty, attendance, examination, results, fees, and library data are maintained under the relevant user roles, while administrators manage the overall system and its records.

The CRUD matrix in the Smart University ERP system represents Create, Read, Update, and Delete operations for different modules. It helps manage student, faculty, attendance, examination, results, fees, and library records efficiently. Each use case specifies how data are accessed and maintained by authorized users. This matrix ensures data consistency, proper access control, and efficient university administration.

Table 8.12: CRUD Matrix

Use Case	STUDENT	FACULTY	ATTENDANCE	EXAM	RESULTS	FEES	Library	ADMIN
Search Student / View Details	R	R						R
Student Registration	C							R
Update Student Details	U							U
Edit Student Profile	U							R
Manage Faculty Details		CRUD						R
Mark Attendance	R	R	CRUD					
View Attendance Reports	R	R	R					R
Manage Examinations		R		CRUD				R

Use Case	STUDENT	FACULTY	ATTENDANCE	EXAM	RESULTS	FEES	Library	ADMIN
Publish Examination Results	R	R		R	CRUD			R
Manage Fee Payments	R					CRUD		R
Manage Library Books	R						CRUD	R
Issue / Return Books	R	R					U	
Manage User Accounts								CRUD
Manage Faculty Accounts		CRUD						CRUD
Generate Reports	R	R	R	R	R	R	R	R

CHAPTER 9 – TOOLS AND TECHNOLOGIES USED

9.1 Overview of Tools and Technologies

The Smart University ERP project is an analysis and design project, so its main tool is a UML modelling tool. StarUML was used to draw all the diagrams of the Smart University ERP model. Microsoft PowerPoint was used for the review presentation, and Microsoft Word was used to prepare this report. The system design is intended for a relational database to store and manage student details, faculty information, attendance records, examination results, fee details, and other university-related information. It does not depend on a particular database product.

Table 9.1 lists the tools and the purpose for which each was used.

Table 9.1: Tools Used in the Project

Tool	Purpose
StarUML	Drawing and storing all the UML diagrams of the model.
Microsoft PowerPoint	Preparing the review presentation of the project.
Microsoft Word	Preparing this project report.
Relational database	Target for the tables designed for student management, faculty management, attendance, examinations, and fee management; the database product is not fixed by the design.

9.2 UML Modelling Tool

StarUML is the UML modelling tool used in this project. The complete Smart University ERP model contains diagrams, including use case, class, sequence, communication, activity, state machine, package, component, and deployment diagrams.

The diagram types that were drawn in the tool and the chapter in which each is used are as follows:

- Use case diagram: Chapter 3.
- Class diagram (domain and design): Chapters 3 and 5.
- Sequence and communication diagrams: Chapter 4.
- Activity and state machine diagrams: Chapter 6.
- Package, component and deployment diagrams: Chapter 7.

9.3 Introduction to the Tool

StarUML is a software modelling tool that supports the UML 2 notation. It allows the designer to create diagrams for the Smart University ERP system and keeps model elements, such as classes, actors, and use cases, in a single model shared by all diagrams. A change to an element can therefore be reflected in every diagram in which it appears. It helps in designing and visualising the university ERP system, including student management, faculty management, attendance, examination, fee management, and library management.

9.4 Features and Capabilities

- Support for the main UML diagram types used in the Smart University ERP project.
- A model explorer in which the elements of the entire university ERP model are organised and can be renamed or moved.
- Properties for each element, such as attributes, operations, visibility, and multiplicity for students, faculty members, administrators, and other system components.
- Export of diagrams as images or documents, which were inserted in this report.
- An extension mechanism, which can add further features such as code generation.
- A single model for all diagrams, which keeps the names and relationships consistent from one diagram to the next.
- Support for stereotypes such as boundary, control and entity, which were used in the sequence diagrams.

Using a modelling tool instead of drawing diagrams by hand saves time when changes are made, because updates can be managed within the model. It also helps maintain consistency and produces neat, clear, and readable diagrams for the Smart University ERP project report.

CHAPTER 10 – CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

This project analysed and designed a Smart University ERP System using OOAD and UML. The requirements were identified as functional and non-functional requirements, including actors such as students, faculty members, administrative staff, examination staff, and administrators, along with their related use cases. The structure of the system was modelled using domain and design class diagrams, the behaviour using sequence, communication, activity, and state machine diagrams, and the architecture using package, component, and deployment diagrams. The class model was mapped to relational database tables.

The model demonstrates inheritance (User and its subclasses), encapsulation (private attributes with public operations), association with multiplicity, and the Boundary–Control–Entity layering. The student registration, attendance management, examination, fee payment, and result management processes are described in detail, and together the models form a sound basis for implementation.

The project also gave the team practical experience in the following areas:

- Identifying actors, use cases and requirements from a university management problem statement.
- Moving from a domain model to a design model and then to database tables.
- Showing the same behaviour in sequence, communication, activity and state diagrams, and checking that they agree.
- Dividing a system into packages, components and nodes so that it can be developed, maintained, and deployed in parts.
- Understanding how an integrated ERP system can simplify university administration by connecting student information, faculty activities, attendance, examinations, fees, and academic records in one system.

10.2 Limitations

- The project covers analysis and design only; the application has not yet been implemented.

- Some modules, such as Library Management, Examination Management, Fee Management, Attendance Management, and Faculty Management, may not yet be fully included in the system model.
- The user role hierarchy covers only some roles, while the system also requires students, faculty, administrators, librarians, and examination staff.
- The technology stack for the implementation has not yet been finalized.
- Some diagrams, such as the system sequence diagram, activity diagram, and high-level sequence diagram, still need to be completed.
- The model has not yet been tested with real university data or actual students, faculty members, and administrative staff.
- Integration with external systems, such as online payment gateways, SMS notifications, and e-mail services, has not yet been implemented.

10.3 Future Enhancements

- Add the missing modules (Student Management, Faculty Management, Attendance, Examination, Library, Fee Management, and Administrator) and extend the system to support all university departments.
- Implement the system, starting with the student registration and management module, and integrate secure online fee payment gateways.
- Add mobile applications and e-mail or SMS notifications for attendance, examination schedules, fee due dates, and university announcements.
- Add reporting and analytics, such as student performance, attendance percentages, fee collection, and faculty workload.
- Add support for multiple departments, courses, academic programs, and university campuses.
- Add online examination registration, hall ticket generation, and digital results publishing for students.
- Add student services such as course registration, timetable management, assignment submission, and online learning resources.
- Add automatic alerts to students and faculty when class schedules change, examinations are rescheduled, attendance is low, or important university announcements are published.